ALL ABOUT AGENTS · SYSTEM DESIGN

A Mental Model for Designing Agentic Systems

A 20-minute design exercise for a self-healing code repository, and the twelve questions that turn three wishes into a design.

By Eva Khmelinskaya · November 2026 · about 15 minutes to read

It is tempting to treat an AI agent as a genie in a bottle. You state the wish, and the result appears: an agent that finds the problem, an agent that fixes it, an agent that ships the fix. We put language models into systems precisely for that flexibility—so we no longer have to anticipate and hardcode every trajectory in advance. As frontier models improve and earn trust in everyday work, handing them a vague goal and watching them figure out the steps becomes routine. The more capable they get, the easier it is to draw each agent as an autonomous black box that just works.

At the same time, the tooling around these models moves fast. New frameworks, SDKs, and APIs keep arriving, and there is always another interface to learn. Underneath that churn, the architectural questions change far more slowly. This post is the checklist I wrote to shift my own mental model and keep myself from treating every agent like a black box that will figure it out on its own—one drawing and twelve questions that I can reuse on any system of this kind.

The Task

This episode is set up as a design exercise that aims for about twenty minutes—intentionally tight, so you have to focus on the right priorities instead of getting lost in details.

> **The task:** Design a self-healing code repository system with three agents: an agent that detects issues, an agent that suggests a fix, and an agent that opens a pull request with the fix. It is expected to handle `10,000` deployments a day across `20` to `30` repositories, of which `1%` fail.

> **Challenge yourself before reading on.** Take a few minutes and think about how you would design this system. What would you draw first? Which questions come to mind, and which would you ask before calling the design finished?

Figure 1 shows the three agents named in the task.

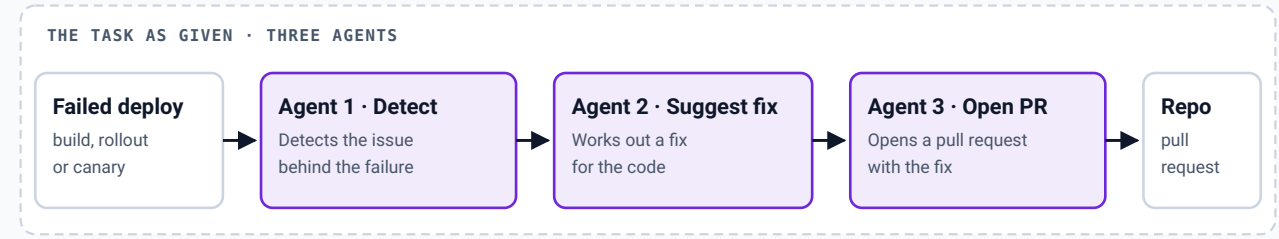


Figure 1 — The task, drawn as given. Agent

The Genie in the Bottle

This is the point where I used to go wrong. Each agent in Figure 1 has a clear job, and the three jobs together cover the task. So the drawing looks finished, and the next step seems to be picking a model and writing three prompts.

Naming what each agent does is not an architecture. Each box is still a sealed bottle, and it is easy to let the model inside stand in for every unanswered question:

- How does the first agent tell a bug in the code from an outage in a shared service? *It figures it out.*
- What if the fix from the second agent is wrong? *It will test it.*
- What does the third agent need from the second? *Whatever the second one found.*

To carry one failure through the whole post, here is the running example.

> **The running example:** At 14:00 the `payments` repository deploys commit `a3f91c2`. A canary instance starts failing with `KeyError: 'vat_rate'` in `payments/tax.py`. The

new code reads a tax rate that is missing for one country. The right fix is a few lines in `tax.py` and a test for the missing case.

THE SKELETON

The Skeleton Most Agent Systems Share

The agents are the small part. Most of the system is the plain machinery around them.

For every box in an architecture diagram, first ask what goes wrong without it. If you cannot name the failure mode it prevents, you do not need the box.

THE RULE THAT ORGANIZES THE WHOLE DESIGN

“The model does the reading and the trying. Code does the deciding, the counting, and the writing.”

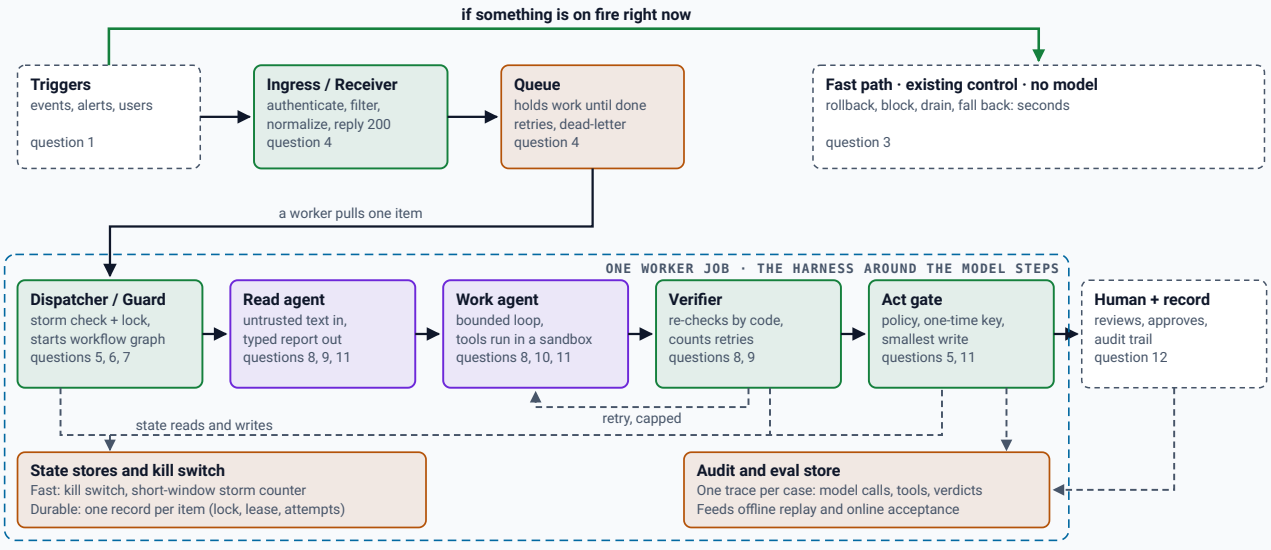


Figure 2A — The generic skeleton: two model steps with code on every side. In the example, Read agent = Agent 1, Work agent = Agent 2, Act gate = Agent 3.

Plain code Model call State / queue Existing control / human

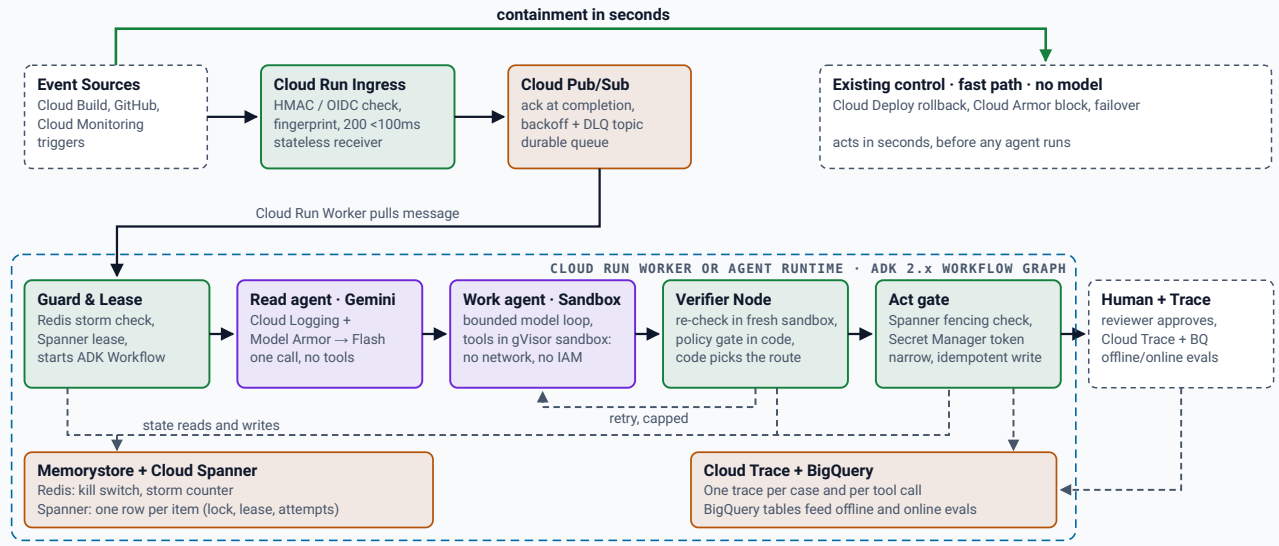


Figure 2B — The same skeleton on Google Cloud. Each box keeps the role it has in Figure 2A.

Plain code Model call State / queue Existing control / human

THE PLAYBOOK

Twelve Questions for Any Agentic System

Ask them in this order. Each card shows what breaks if you skip it, what to decide, and how it applies to the self-healing repo.

BEFORE DRAWING

1 What wakes the system up, and what is in the wake-up message?

SKIP IT AND…

- You design for one trigger and miss two others.
- Every agent has to understand every sender's format.
- Someone puts 50 MB of logs inside a queue message.

DECIDE

- List every source that can start the work.
- Normalize into one common message: id, source, subject, exact version, duplicate key, and a *pointer* to the data.

IN THE SELF-HEALING REPO

- Sources: CI, the CD controller, and post-deploy monitoring.
- `FailureEvent(repo, commit_sha, failure_fingerprint, log_pointer)`.

2 How big is it really?

SKIP IT AND…

- You spend time sizing model capacity for the headline number.
- You optimize for throughput when the real problem is correctness.

DECIDE

- Multiply headline volume by failure rate to get events per day and per hour.
- Name what kind of problem that makes it: scale, or correctness and safety.
- Estimate cost per case, cache static instructions, and escalate to stronger models or higher thinking only on retries.

IN THE SELF-HEALING REPO

- 10,000 × 1% = 100 failures a day, about 4 an hour.
- Attempt 1 uses a fast model (`LOW` thinking) with cached repo rules; only RETRY escalates.

3 What must happen in seconds, and who does it?

SKIP IT AND…

- Users wait minutes while a model diagnoses the bug.
- The agent ends up holding control of live production traffic.

DECIDE

- Find the fast, model-free control that already exists: rollback, block, rate-limit, fallback.
- Keep containment in existing tooling. The agents work on a second clock afterwards.

IN THE SELF-HEALING REPO

- The CD controller halts the canary and rolls back in seconds.
- The agents repair the code afterwards and never touch live traffic.

THE PLUMBING

4 How does work get from the event to a worker without being lost?

SKIP IT AND…

- The sender times out waiting for a slow model run.
- A burst drops events.
- Anyone can post fake events to the endpoint.

DECIDE

- Put a small receiver in front: verify the sender, filter noise, reply at once.
- Put a queue between receiver and worker; acknowledge only at completion.
- Retry with exponential backoff (model quota errors included) and dead-letter after N failures.

IN THE SELF-HEALING REPO

- Webhook receiver → Pub/Sub → worker job.
- Verify signature, reply 200 in under a second, dead-letter after 5 tries.

5 How do I avoid doing the same thing twice?

SKIP IT AND…

- Fifteen identical pull requests, tickets, or refunds.
- A retry repeats a write that already succeeded.

DECIDE

- Compute a deterministic key in code from stable fields—never by a model.
- Keep one durable record per key with an explicit state.
- Check the record before working, and pass an idempotency key on the write.

IN THE SELF-HEALING REPO

- `failure_fingerprint`: hash of error type, code location, and cleaned message.
- One `incidents` row per `(repo, failure_fingerprint)`: `IN_PROGRESS`, `PR_OPENED`, `ESCALATED`.

6 How do I tell one problem from everything being on fire?

SKIP IT AND…

- A shared registry or base-image outage makes every worker "fix" healthy code.
- Token budget burns and humans get flooded.

DECIDE

- Maintain a shared, short-window count of *distinct* things failing.
- Above a threshold, trip a circuit breaker, park incoming work, and alert once.
- Provide a manual kill switch, and let one probe through after cool-off.

IN THE SELF-HEALING REPO

- Redis set of distinct repos failing in the last 5 minutes.
- 5 or more repos → pause PR creation for 15 minutes and page on-call.

7 What happens when a worker dies halfway?

SKIP IT AND…

- Acknowledged on receipt: the work is lost when the worker crashes.
- Locked with no expiry: the work stays stuck forever.

DECIDE

- Acknowledge the queue message only after the final step succeeds.
- Claim work with a lease that expires, and renew it on a heartbeat while alive.
- Right before any external write, re-check ownership (fencing) and that the target has not moved on.

IN THE SELF-HEALING REPO

- Pub/Sub redelivery if the worker dies.
- 120-second lease in the incident row, renewed every 30 seconds; fencing check before opening the PR.

8 Which steps need a model, which are code, and what is the harness?

SKIP IT AND…

- "Three models chatting": unpredictable, unbounded, impossible to audit.
- A model grades its own work (and agrees with its own mistakes).
- An agent runs on a host with credentials and cannot isolate itself.

DECIDE

- Give each step the least autonomy that works (four-row test). If each of ten model steps is right 95% of the time, all ten are right about 60% of the time.
- Wrap the model steps in a **harness**: the code around the model that owns state, what each model sees, retry counters, secrets, and sandboxes.
- Fix the control flow as a graph when every case follows the same steps or a mistake is costly. Let the model choose its steps only for open-ended, supervised work.

IN THE SELF-HEALING REPO

- **Agent 1 (Detect):** rules + one schema-bound call, no tools.
- **Agent 2 (Fix):** bounded loop. Its tools run in Sandbox 1 (no network, no IAM).
- **Verifier and Agent 3 (Open PR):** plain code. The verifier also rejects a patch that deletes or skips tests.

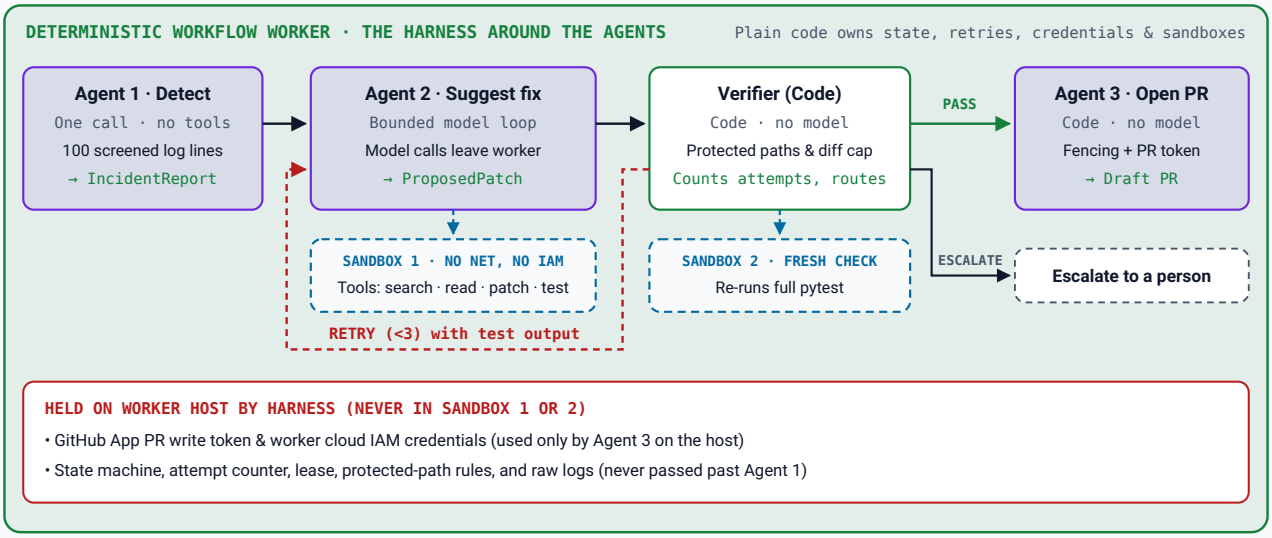


Figure 3 — Inside the harness for the example. Agent 3 is named an agent by the task and contains no model.

Harness / Code Agent named in the task Isolated sandbox

```
The same graph sketched with ADK 2.x Workflow (agent and helper definitions omitted)

from google.adk import Agent, Context, Event, Workflow

def verify_patch_in_sandbox(ctx: Context, patch: ProposedPatch) -> Event:
    if touches_protected_path(patch.target_files):
        return Event(route="ESCALATE", output={"reason": "protected_path"})
    result = run_sandbox_tests(patch.unified_diff)  # --network=none, 0 creds
    if result["passed"]:
        return Event(
            route="PASS",
            output={"patch": patch, "stdout": result["stdout"]},
        )
    attempts = ctx.state.get("fix_attempts", 0) + 1
    ctx.state["fix_attempts"] = attempts
    if attempts < 3:
        return Event(route="RETRY", output={"stderr": result["stderr"][:2000]})
    return Event(route="ESCALATE", output={"reason": "max_attempts_exhausted"})

self_healing_workflow = Workflow(
    name="self_healing_code_repo",
    edges=[
        ("START", dedup_and_circuit_breaker),
        (dedup_and_circuit_breaker, {
            "ANALYZE": detector_agent,
            "SKIP": skip_node,
            "ESCALATE": escalate_node,
        }),
        (detector_agent, route_incident),              # typed IncidentReport handoff
        (route_incident, {"FIX": fixer_coding_agent, "ESCALATE": escalate_node}),
        (fixer_coding_agent, verify_patch_in_sandbox), # typed ProposedPatch handoff
        (verify_patch_in_sandbox, {
            "PASS": open_draft_pr,                    # Agent 3: plain code + PR token
            "RETRY": fixer_coding_agent,              # code-owned back-edge
            "ESCALATE": escalate_node,
        }),
    ],
)
```

What Is Deployed

The three agents are not three deployments. They are nodes of one workflow graph inside one worker service. The guard is the first node: plain code that decides whether this case should run at all. Agent 1 and Agent 2 are model nodes. Agent 3 is a function node, and so are the guard, the verifier, and the escalation step. One queue message starts one run of the graph, and the message is acknowledged when the run ends.

Agent 2's model calls leave from the worker, which can reach the model endpoint. Only its tools run in the sandbox, which has no network and no credentials.

A node becomes its own service only for a reason: it needs a different identity or network boundary, it scales differently, or another team owns it. The typed contract between the two stays the same.

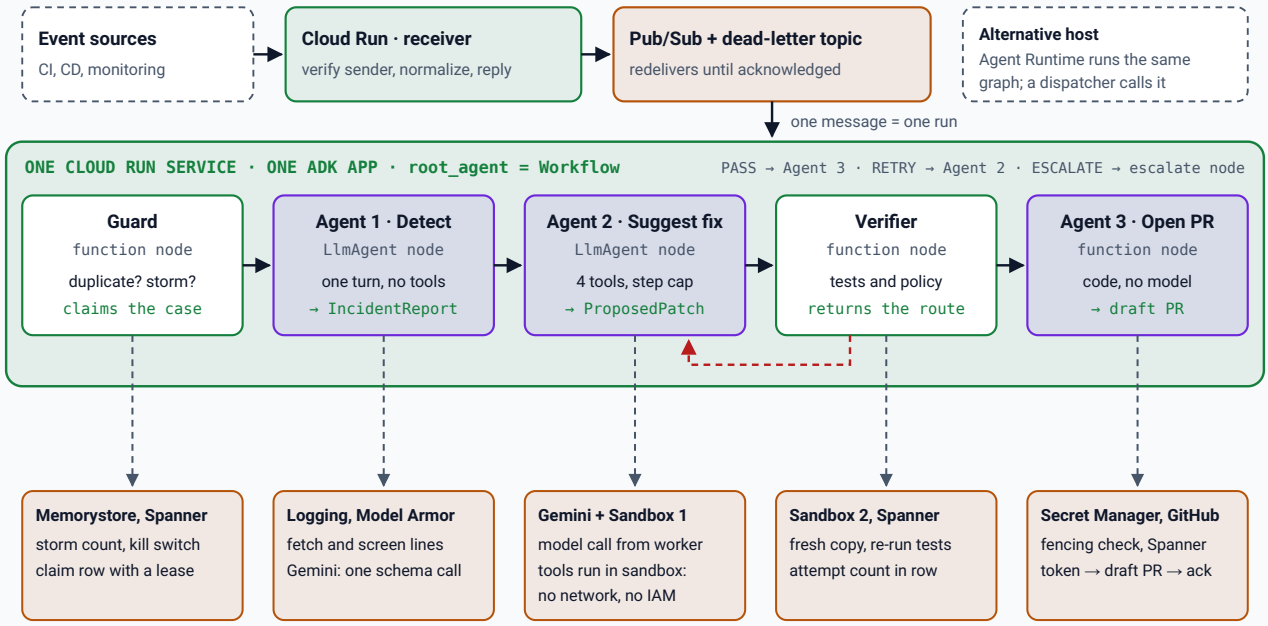


Figure 4 — The example deployed on Google Cloud: one worker service, one workflow graph, and what each node calls.

▢ Agent named in the task ▢ Function node ▢ Google Cloud service

STEP	KIND	RECEIVES	WHAT IT DOES	TOOLS AND SERVICES IT CALLS	RETURNS
Receiver	Separate Cloud Run service, no model	Webhook from CI, the CD controller, or monitoring	Verifies the signature or identity token. Drops healthy deployments. Computes `failure_fingerprint` from error type, code location, and cleaned message. Replies at once.	Pub/Sub publish	`FailureEvent(repo, commit_sha, failure_fingerprint, log_pointer)`

STEP	KIND	RECEIVES	WHAT IT DOES	TOOLS AND SERVICES IT CALLS	RETURNS
Guard	Function node	`FailureEvent`	Reads the kill switch. Adds the repo to the five-minute storm set and counts distinct repos. Looks up the incident row for `(repo, failure_fingerprint)`: if a pull request is open or another worker holds a live lease, it stops. Otherwise it claims the row with a 120-second lease, renewed every 30 seconds.	Memorystore (Redis) for the kill switch and storm set. Spanner transaction on the incident row.	Route: `ANALYZE`, `SKIP`, or `ESCALATE`
Agent 1 · Detect	`LlmAgent` node, one turn	A pointer to the logs, resolved by worker code into at most 100 screened error lines	Classifies the failure and names the suspected files. Treats log text as data. May answer `flaky` or `unknown`.	Tools: none. Worker code before the call: Cloud Logging query by deployment id, Model Armor screening. Model: Gemini with a fixed output schema.	`IncidentReport(category, suspected_files, failing_test, confidence)`
Agent 2 · Suggest fix	`LlmAgent` node, bounded loop	`IncidentReport`. On a retry, also the verifier's test output, truncated.	Reproduces the failure, reads around it, applies the smallest patch, runs the tests. Stops at a step cap and a time cap.	Tools, all executing in Sandbox 1 at the failing commit: `ripgrep_search(pattern, path)`, `read_file_lines(path, start, end)`, `apply_patch(unified_diff)`, `run_sandbox_tests()`. Model: Gemini, called from the worker, with a stronger setting on retries.	`ProposedPatch(target_files, unified_diff, explanation)`
Verifier	Function node, no model	`ProposedPatch`	Applies the diff to a fresh copy and re-runs the full test suite. Rejects protected paths, oversized diffs, and deleted or skipped tests. Increments the attempt counter.	Sandbox 2. The attempt counter is written by code: workflow state in the sketch above, the incident row if it has to survive a worker crash.	Route: `PASS`, `RETRY` (fewer than 3 attempts), or `ESCALATE`
Agent 3 · Open PR	Function node, no model	The verified patch and the verifier's test output	Checks that this worker still owns the incident row. Pushes a branch, opens a draft pull request with the evidence, requests review from the code owners, records `PR_OPENED`. The run ends and the message is acknowledged.	Spanner for the ownership check and final state. Secret Manager for the GitHub App token. GitHub API, limited to `autofix/*` branches.	Draft pull request
Escalate	Function node, no model	The reason and what was tried	Opens an issue for a person, records `ESCALATED`. No pull request.	GitHub issue or pager. Spanner for the final state.	Issue for a person

9 What passes between the agents, and who decides what is next?

SKIP IT AND…

- Each agent reinterprets the previous one's prose; errors compound.
- Injected text travels straight into the agent that can act.
- A retry loop has no exit.

DECIDE

- Use typed contracts with a few validated fields—never raw chat history.
- Keep routing (PASS, RETRY, ESCALATE) and retry counters in code.
- If subtasks run in parallel, keep helper subagents read-only and leave writes to one owner.

IN THE SELF-HEALING REPO

- `FailureEvent` → `IncidentReport` → `ProposedPatch`.
- The verifier returns one of three routes; capped at 3 attempts.
- Agent 1 may answer `flaky` or `unknown`: no patch, escalate.

10 Where does the data live, and how does the agent read it?

SKIP IT AND…

- A pre-built copy goes stale or drops access rules.
- The model gets flooded with whole files or noisy stack traces.
- Models poison shared long-term memory across runs.

DECIDE

- Leave data where it lives; read by pointer with strict bounds (version, time window, line range).
- Read with the identity whose permissions should apply, and give each component the least it needs.
- Keep tools narrow with bounded output. Operational state is written by code, not by models.

IN THE SELF-HEALING REPO

- Repo checked out at exact failing `commit_sha` (no stale search index).
- Pointer tools (`ripgrep_search`, `read_file_lines`) and truncated test output; 100 error lines max.

11 What would an attacker do with it?

SKIP IT AND…

- Text an outsider controls reaches a model that can run tools.
- Model-written code runs next to real cloud credentials.
- Write permissions are wider than the job requires.

DECIDE

- Mark where untrusted input enters; give that reader no tools.
- Run generated code in a sandbox with no network and no credentials.
- Keep write permission narrow and enforced in code. Irreversible actions wait for a person.

IN THE SELF-HEALING REPO

- Webhook signature, content screening, and toolless Agent 1.
- Sandboxed tests, protected-path check, GitHub App limited to `autofix/*`, draft PRs.

12 How do I switch it on, and how do I know it is working?

SKIP IT AND…

- A noisy bot is muted by engineers within a week.
- You have no evidence that it is safe to grant more autonomy.

DECIDE

- **Offline evals:** run a golden replay set before launch and on every prompt, model, or tool change (track pass@1 vs. pass@3).
- **Online evals & rollout:** launch in stages (shadow → drafts → bounded) and track live human acceptance with automatic demotion.
- Keep one trace per case: every model call, tool call, and verdict.

IN THE SELF-HEALING REPO

- Offline replay gates every change against historical fixes.
- Online: shadow → draft PRs → ready-for-review; watch PRs merged without edits and human diff-edit distance.

CONTRACTS & STATE

What Passes Between Agents, and Who Owns State

Typed contracts at every boundary, what stays behind, and where short-term and long-term state live.

Typed Handoffs (and What Stays Behind)

Each handoff is a small typed contract validated by workflow code on exit and on entry. Anything that does not fit the schema is rejected or retried before any side effect runs.

RECEIVER → WORKER

FailureEvent

```
{
  "repo": "acme/payments",
  "commit_sha": "a3f91c2",
  "failure_fingerprint": "fp_8d4e",
  "log_pointer": "dep_4471"
}
```

A pointer to the logs, not the 50 MB log payload.

AGENT 1 → AGENT 2

IncidentReport

```
{
  "category": "test_regression",
  "suspected_files": ["payments/tax.py"],
  "failing_test": "tests/test_tax.py",
  "confidence": 0.86
}
```

Raw logs stop here. Agent 2 never sees log text or Agent 1's prompt.

AGENT 2 → VERIFIER → AGENT 3

ProposedPatch

```
{
  "target_files": ["payments/tax.py"],
  "unified_diff": "--- a/… +++ b/…",
  "explanation": "Default missing vat_rate"
}
```

Agent 3 receives it only after the verifier's own test run passes.

Who Owns Which State

Operational state is written by workflow code and never by a model, so one bad run or one injected instruction cannot poison the next run.

STATE	WHERE IT LIVES	WHO WRITES IT	WHY
Storm counter and kill switch	Shared short-window store (e.g. Redis)	Workflow code only	Counts distinct failing repos over 5 minutes; trips the circuit breaker at 5 or more. A person can flip the kill switch.

STATE	WHERE IT LIVES	WHO WRITES IT	WHY
Incident lock, lease & attempt count	Durable incident row per `(repo, failure_fingerprint)`	Workflow code only	Prevents duplicate PRs, expires dead worker claims, and caps attempts at 3.
Loop working history	In-memory trace inside Sandbox 1	Agent 2 loop (ephemeral)	Tracks files read and test output for one attempt; destroyed with the sandbox.
Audit trace & replay cases	Append-only log / eval store	Harness on every call	Records every prompt, tool call, and verifier exit for debugging and regression replay.

RULES OF THUMB

Model or Code? And How the Skeleton Transfers

A four-row test for every step you draw, and two other systems built on the same skeleton.

Model or Code? A Four-Row Test

Apply this test to every box in the diagram. Use the lowest level of autonomy that solves the step.

IF THE STEP…	BUILD IT AS	LIMITS TO STATE	IN THE SELF-HEALING REPO
follows a rule	Plain code	None needed. Same input, same result.	Filter, duplicate key, storm count, path rules, retry counter, opening the PR
must understand messy text	One model call	No tools. Output must fit a fixed schema. Bounded input.	Turn 100 screened error lines into an `IncidentReport`
must try things to find out	Bounded model loop + code check	Step cap, time cap, isolated sandbox. Code re-checks the result.	Reproduce, patch, test; verifier re-runs full suite in a clean sandbox
changes the outside world	Plain code + human	Narrow permission, idempotency key, size limits, review.	Draft PR on a restricted `autofix/*` branch

The Same Skeleton on Other Problems

The questions stay the same across domains; only the nouns change.

	SELF-HEALING REPO	WEATHER ALERT → REROUTE SHIPMENTS	RISK REVIEW OF 100,000 BANK CLIENTS
Wakes it up	CI, CD controller, monitoring	Weather alert feed, carrier status	Nightly feed of clients whose data changed

	SELF-HEALING REPO	WEATHER ALERT → REROUTE SHIPMENTS	RISK REVIEW OF 100,000 BANK CLIENTS
Real size	100 failures a day	Few alerts, each touching thousands of shipments	Only the changed clients, not all 100,000
Fast, no model	Rollback by the CD controller	Rule-based hold on departures in the alert area	None needed; batch workload
Duplicate key	`(repo, failure_fingerprint)`	`(alert_id, shipment_id)`	`(client_id, review_period)`
Everything on fire?	5 repos fail in 5 minutes	Broken feed flags every region at once	Broken data feed flags thousands at once
Model steps	Read logs; try a fix	Read the alert; propose reroute options	Extract facts from filings; draft narrative
Code steps	Verify tests; path rules; open PR	Cost/capacity limits; booking with idempotency key	Financial ratios in code; route to review tiers
Human	Reviews and merges the draft PR	Approves reroutes above a cost threshold	Signs off by risk tier

SUMMARY

One-Page Cheat Sheet

The rule, the order, and the twelve questions in a few words each.

THE RULE

The model does the reading and the trying. Code does the deciding, the counting, and the writing.

The order is the path of one event through Figure 2A: it arrives (1 to 3), survives the plumbing (4 to 7), passes through the agents (8 to 10), and is allowed to act (11 and 12). For every box, name what breaks without it.

BEFORE DRAWING

- **1 · Wake-up.** Every trigger. One normalized message with a pointer to the data.
- **2 · Size.** Volume × rate. Scale problem or correctness problem? Cost per case.
- **3 · Seconds.** An existing, model-free control contains. The agents work afterwards.

THE PLUMBING

- **4 · Intake.** Receiver, queue, acknowledge at the end, backoff, dead-letter.
- **5 · Twice.** Key computed in code, one record per key, idempotent write.
- **6 · Storm.** Count distinct failures, circuit breaker, kill switch.
- **7 · Crash.** Expiring lease, heartbeat, re-check before the write.

THE AGENTS AND THE HARNESS

- **8 · Model or code.** Least autonomy that works. The harness owns state, context, retries, secrets, sandboxes.
- **9 · Handoffs.** Typed contracts. Code routes: pass, retry, escalate.
- **10 · Data.** Read in place, by pointer, bounded. Least-privilege identity. Code writes the state.

SAFETY AND LAUNCH

- **11 · Attacker.** The reader of untrusted text has no tools. Sandbox. Narrow writes. A person on irreversible actions.
- **12 · Launch.** Replay set, then shadow, draft, bounded. One acceptance number, automatic demotion, a trace per case.

In a design session of about twenty minutes: questions 1 to 3 first, then draw Figure 2A and walk one event through it (Q4, Q8, Q9), go deep on two of Q5, Q6, Q7, or Q11, and finish with Q12.

SOURCES

References

External sources on agentic system design, and the related episode.

- Cemri et al., Why Do Multi-Agent LLM Systems Fail? (arXiv:2503.13657). A taxonomy of failures in multi-agent systems, built from annotated execution traces.
- Anthropic, Building Effective Agents. The distinction between workflows and agents, tool interface design, and the case for starting simple.
- Anthropic, How We Built Our Multi-Agent Research System. Effort scaling, context isolation across subagents, and production reliability lessons.
- Google Cloud, Choose a Design Pattern for Your Agentic AI System. Selecting single-agent, workflow, loop, or multi-agent patterns from task shape, latency, and budget.
- Cognition, Don't Build Multi-Agents. Why parallel writer swarms fail from fragmented context, and keeping writes single-threaded.
- HumanLayer, 12-Factor Agents. Engineering principles for production agents: owning your context window, structured tool outputs, unified state, and explicit control flow.
- Episode 01 — Securing the Agent-to-Database Connection. The identity, tool boundary, content screening, and audit controls behind questions 10 and 11.